SECTION 02 · DIMENSIONALITY REDUCTION

Dimensionality Reduction on MNIST

From 784 pixels to a handful of numbers: linear projections, learned parts and 2D maps of the digits.

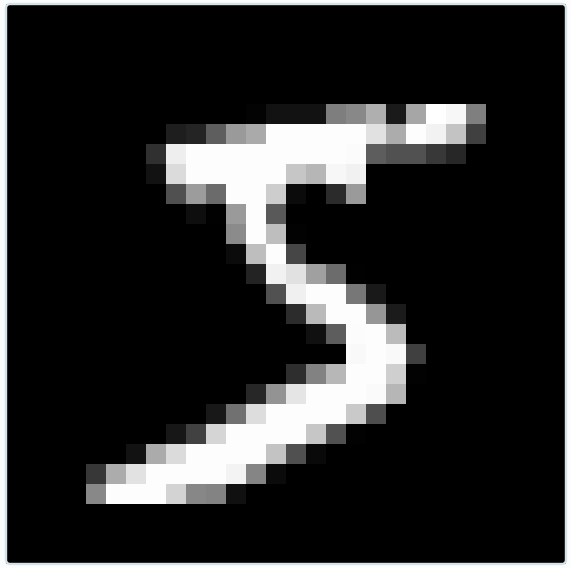
PCA | UMAP | Kernel PCA | Random Projection | ICA | t-SNE

KGP Talkie

Scan to subscribe

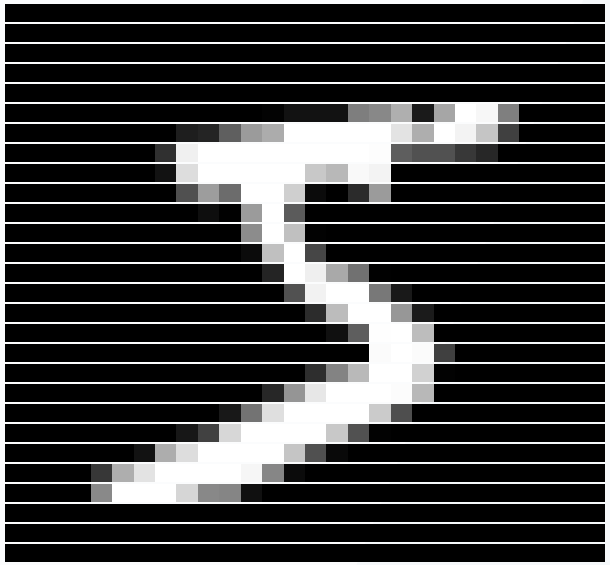
Laxmi Kant Tiwari · kgptalkie.com

A Digit Is a Point in 784 Dimensions



28 × 28 pixels

each one 0 (black) to 1 (white)



row 1, row 2, ... row 28

one long list of 784 numbers

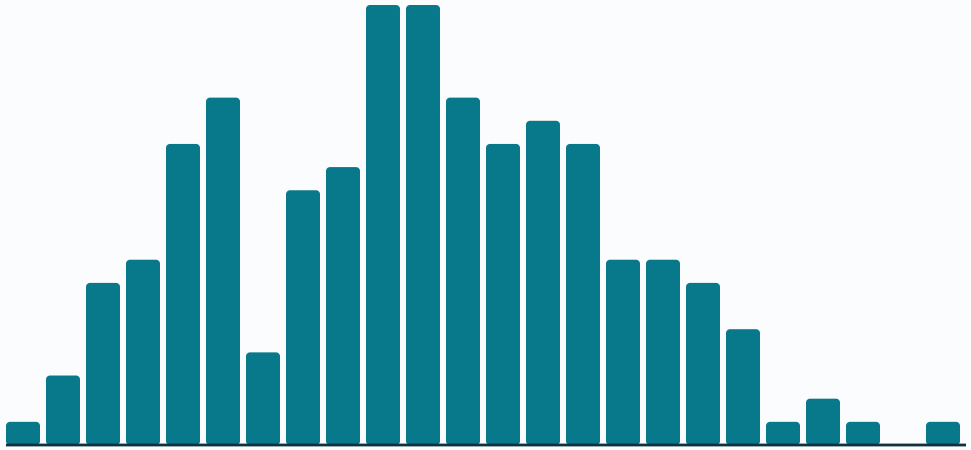
one point

in 784-dimensional space

70,000 digits =
70,000 points

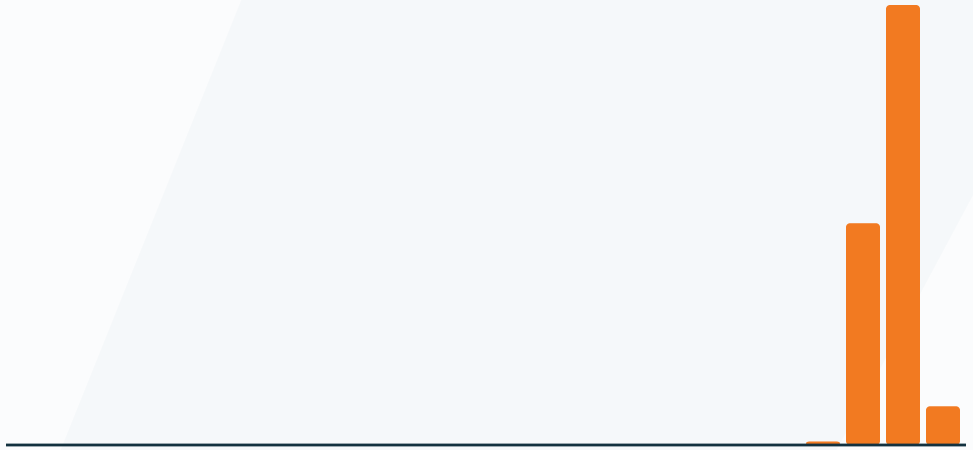
The Curse of Dimensionality

2 dimensions



Distances spread out:
near and far mean something

784 dimensions

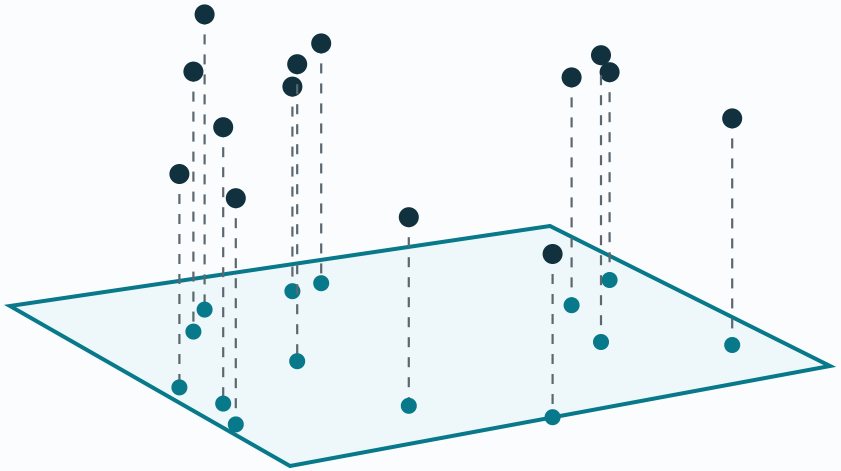


Distances bunch together:
every point looks equally far

Fewer dimensions: faster models, less noise, plots you can see, and distances that mean something.

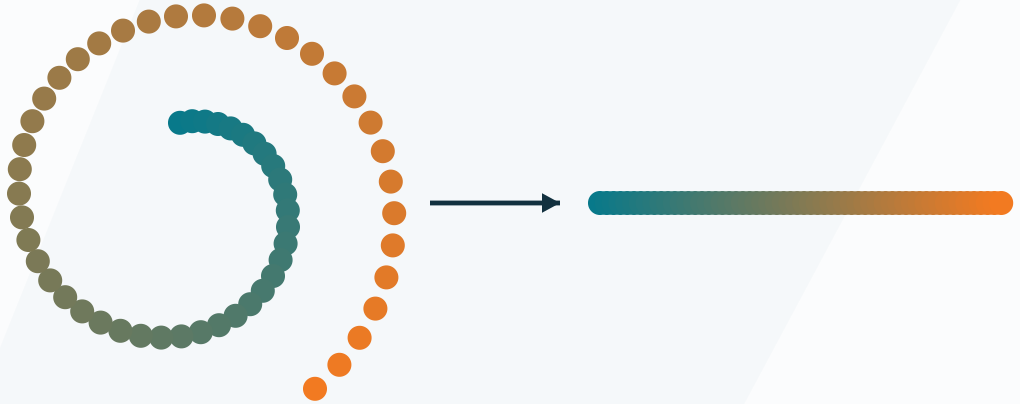
Linear Projection vs Manifold Learning

Linear projection

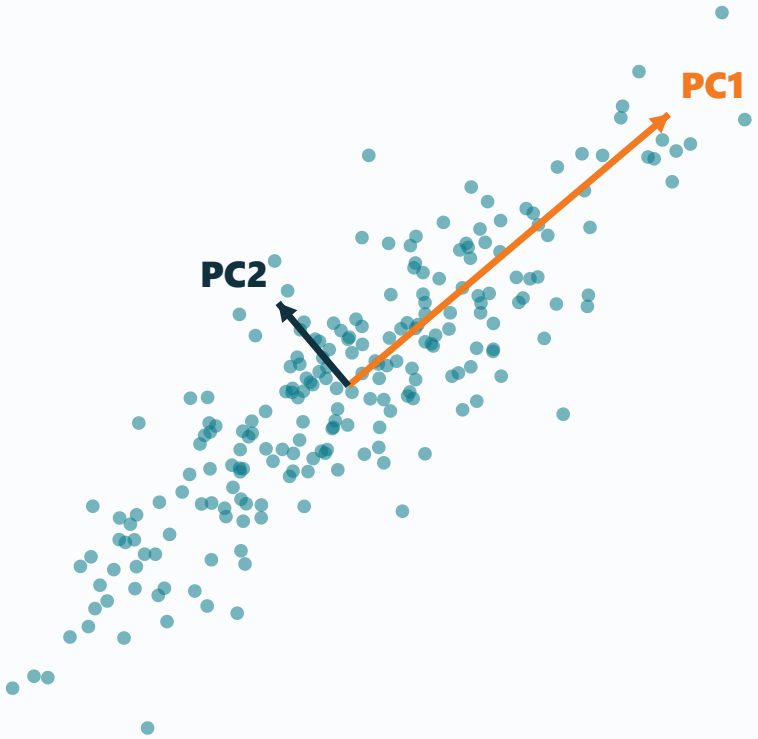


drop each point straight onto
a flat plane through the data

Manifold learning



PCA Finds the Directions of Most Spread



two correlated features, one cloud

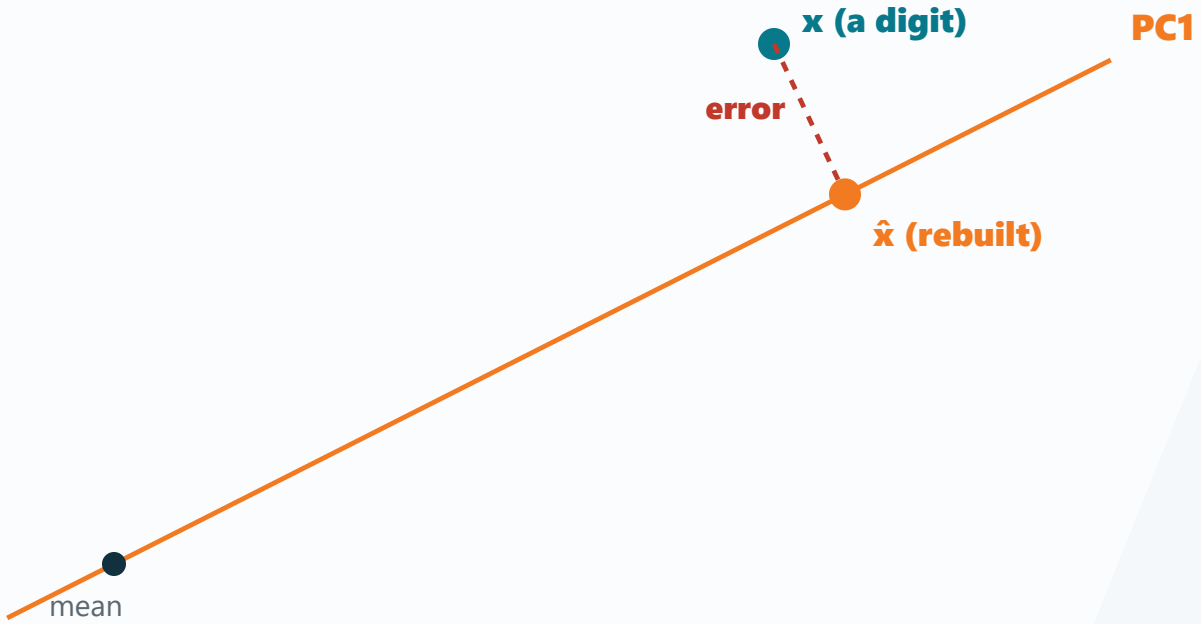
PC1: the direction where the points spread the most

PC2: the most spread left over, at a right angle to PC1

Spread (variance) is treated as information.
Directions with little spread are dropped.

```
components_ = the directions
explained_variance_ratio_ = their share
```

Projecting and Rebuilding a Point



transform

`z = (x - mean) · components`

inverse_transform

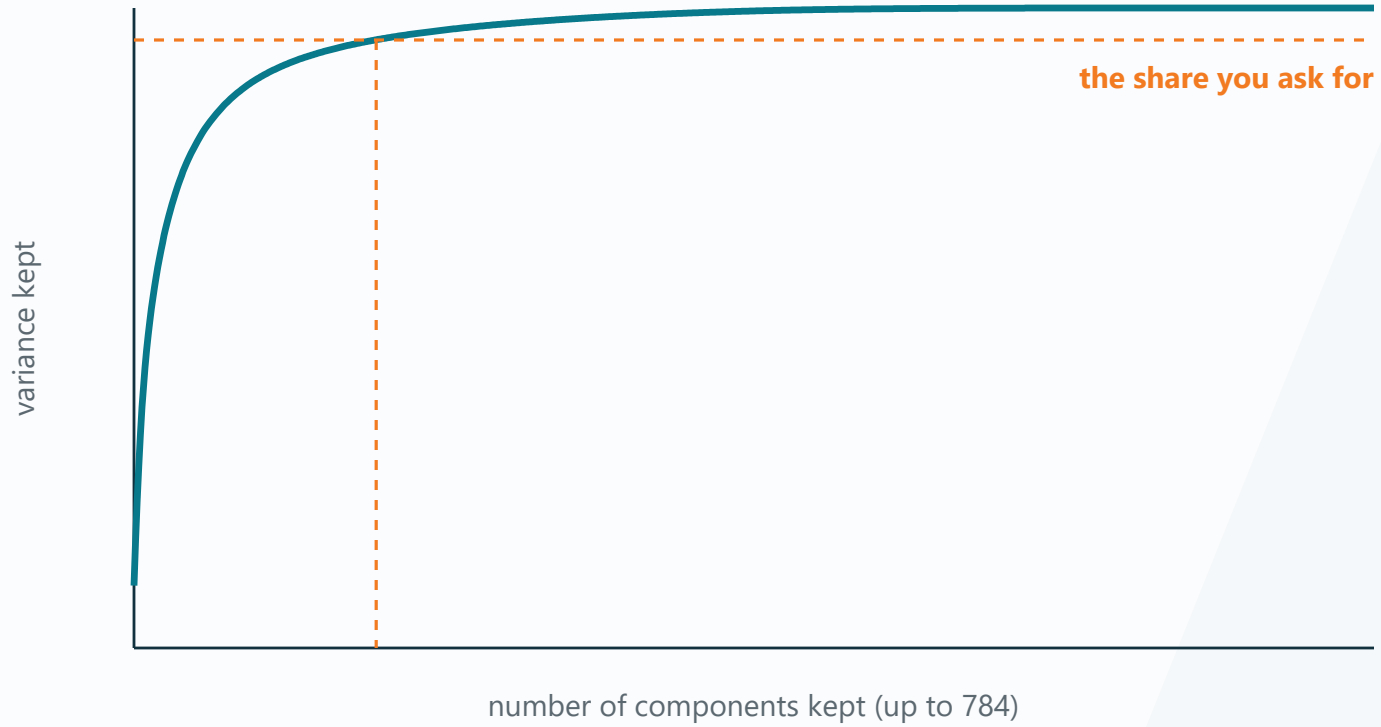
`x̂ = mean + z · components`

reconstruction error

$\| x - \hat{x} \|^2$ summed over pixels

CUMULATIVE EXPLAINED VARIANCE TELLS YOU WHERE TO STOP

How Many Components to Keep



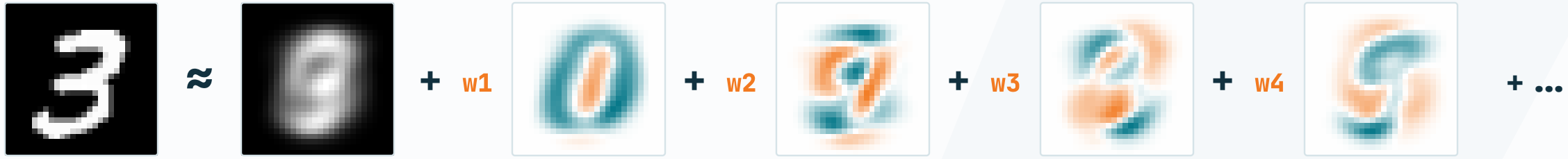
A few components hold most of the spread

The curve rises fast, then flattens: late components add little.

```
PCA(n_components=0.95)
```

a float asks for a share;
an int asks for a count

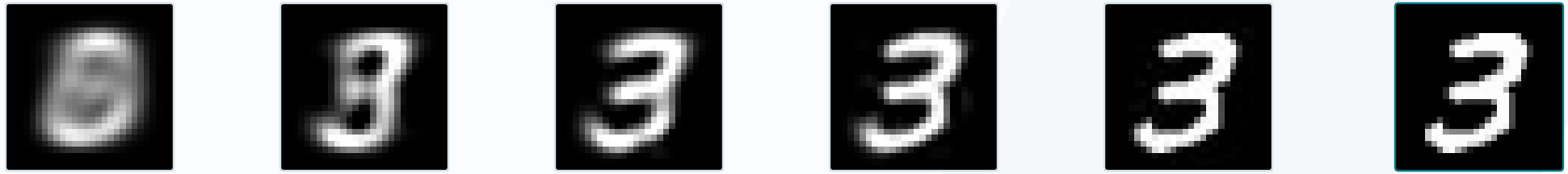
Digits as Mean Plus Components



digit | mean | component 1 | component 2 | component 3 | component 4

Components are patterns: orange where pixels go down, teal where they go up

More components, sharper rebuild:

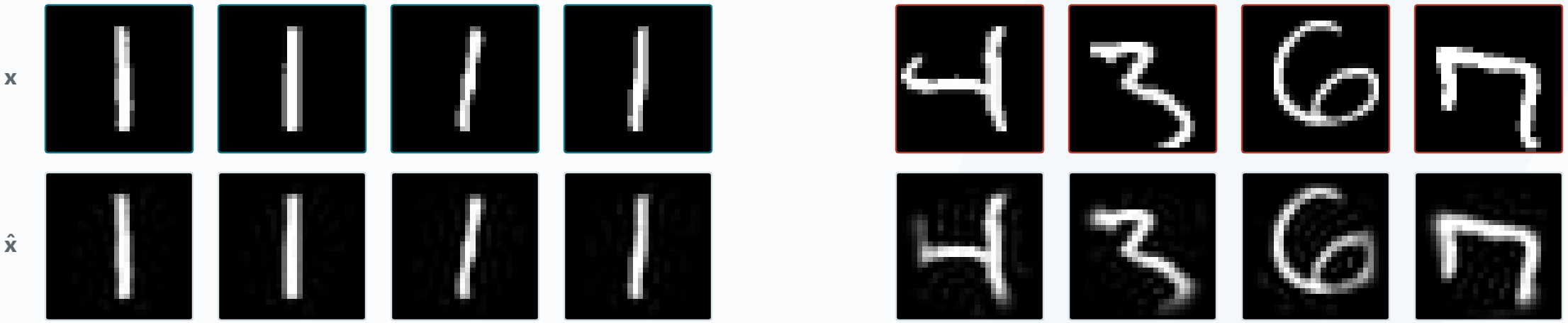


2 components | 10 components | 30 components | 80 components | 300 components | **original**

Reconstruction Error Flags Unusual Digits

Smallest error: rebuilt almost perfectly

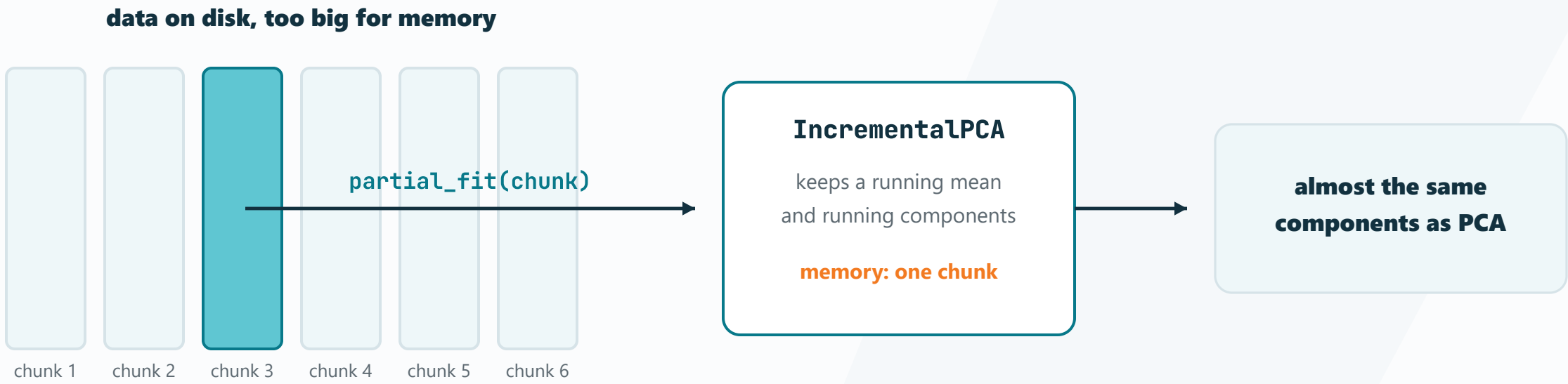
Largest error: unlike most digits



PCA learns what normal looks like. A high error means "not normal".

The upcoming section turns this into a fraud detector.

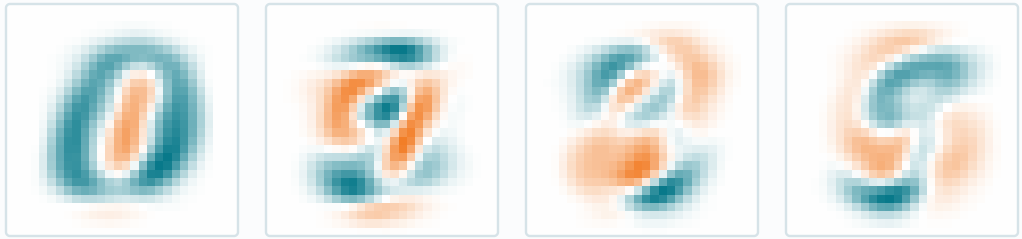
Incremental PCA for Data That Does Not Fit



Use it when the data streams in or does not fit in RAM. Plain PCA needs every row at once.

Sparse PCA: Fewer Pixels per Component

PCA: every pixel takes part



dense weights: hard to say what a component "is about"

Sparse PCA: most pixels are exactly 0

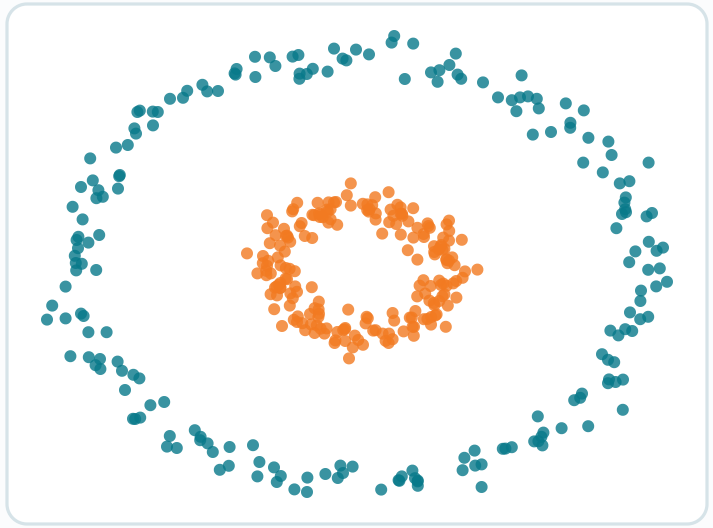


an L1 penalty pushes small weights to 0, so each component covers one region

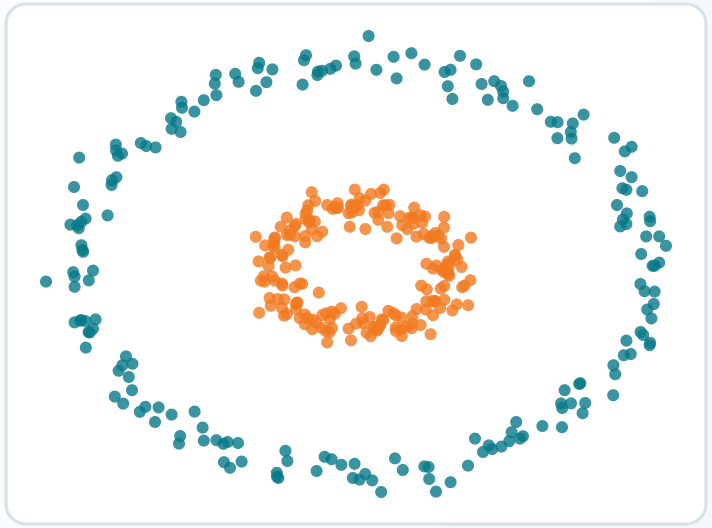
Trade-off: easier to read, but less variance kept per component, and much slower to fit, so fit on a sample and transform the rest.

Kernel PCA and the Kernel Trick

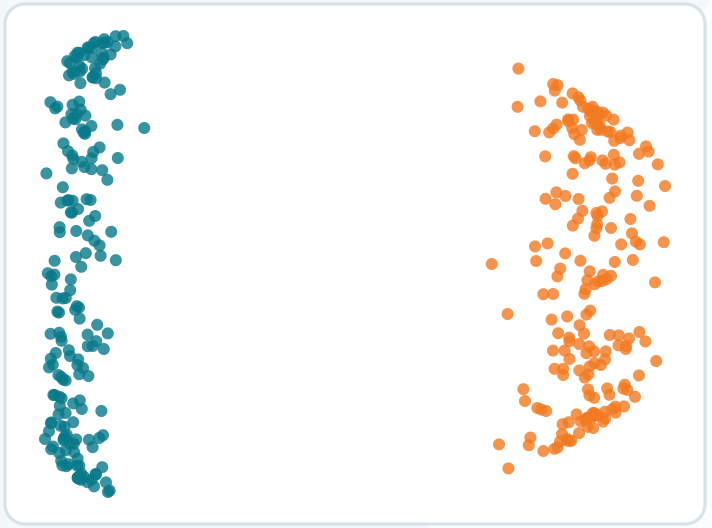
original 2D data



PCA: only rotates it



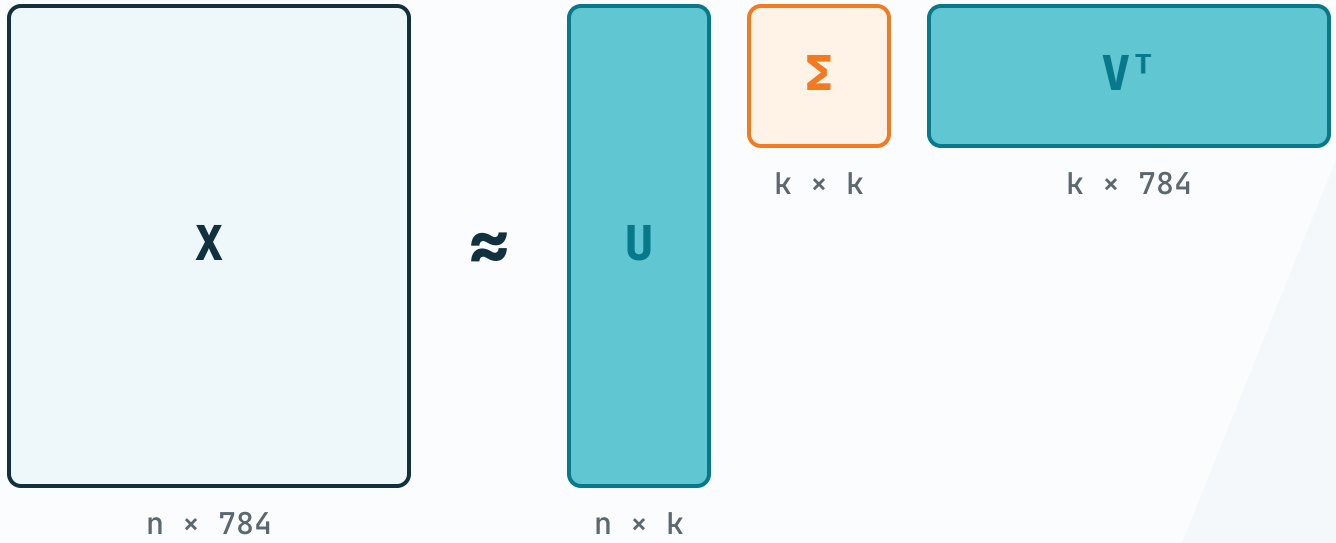
Kernel PCA (RBF): a straight line splits them



$k(a, b) = \exp(-\gamma \|a - b\|^2)$: similarity of every pair, an $n \times n$ matrix

PCA on that matrix finds curved directions. Its size grows with n^2, so fit on a few thousand rows.

Truncated SVD for Sparse Data



fit_transform returns U Σ: k numbers per row

PCA = SVD of centred data

(the mean is subtracted first)

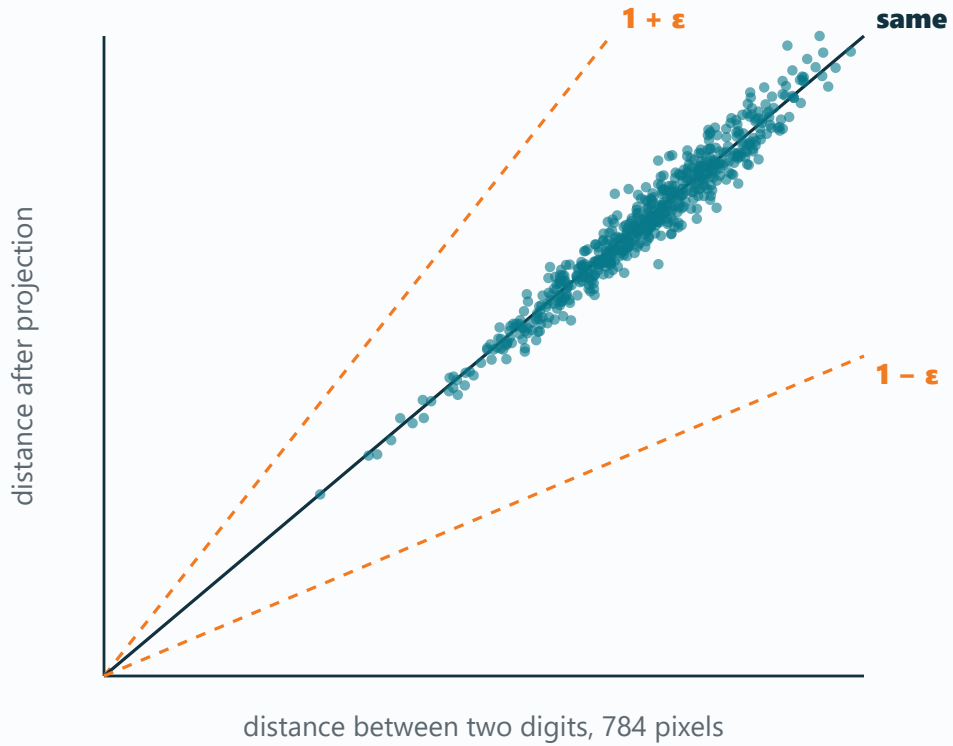
TruncatedSVD skips centring

Subtracting the mean would fill every zero in a sparse matrix with a number.

Use it on sparse data:

word counts, TF-IDF, ratings

Random Projection Keeps Distances



Multiply by a random matrix

`X (n × 784) · R (784 × k) → n × k`

Johnson-Lindenstrauss: with enough columns, every distance stays within (1 ± ε) of the original

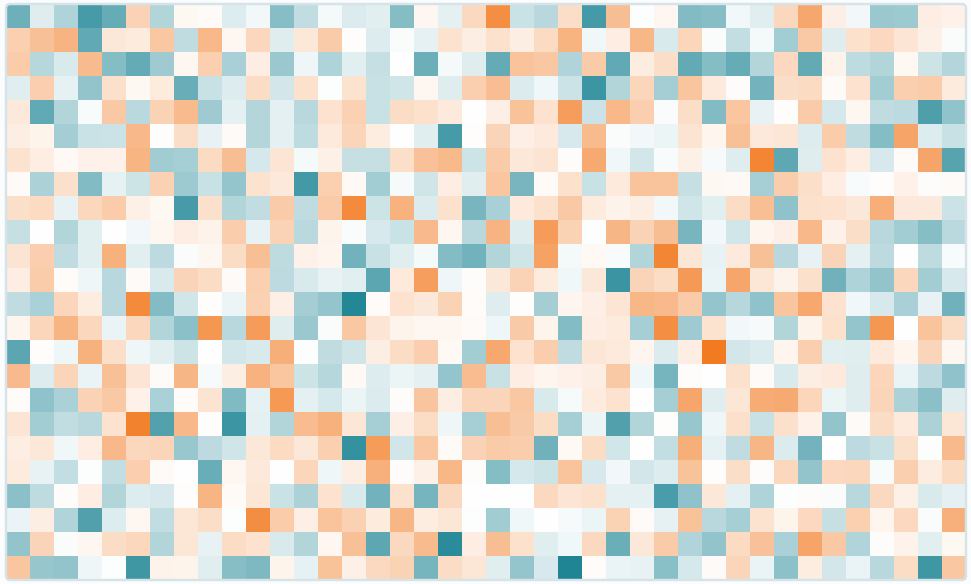
k depends on the number of rows and ε,
not on the 784 input features

No training on the data: fit only draws R.

Fast, but the new axes mean nothing on their own.

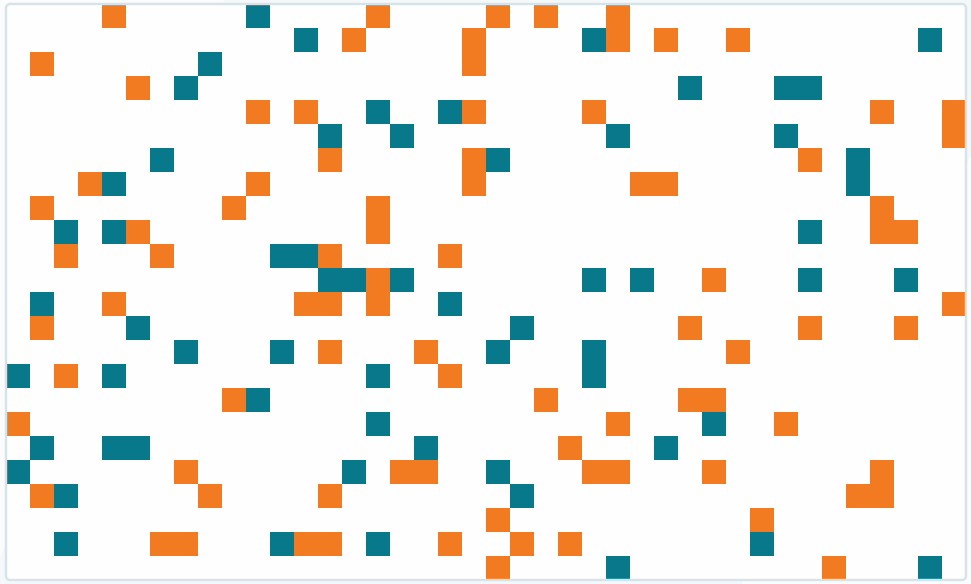
Gaussian vs Sparse Random Matrices

Gaussian random projection



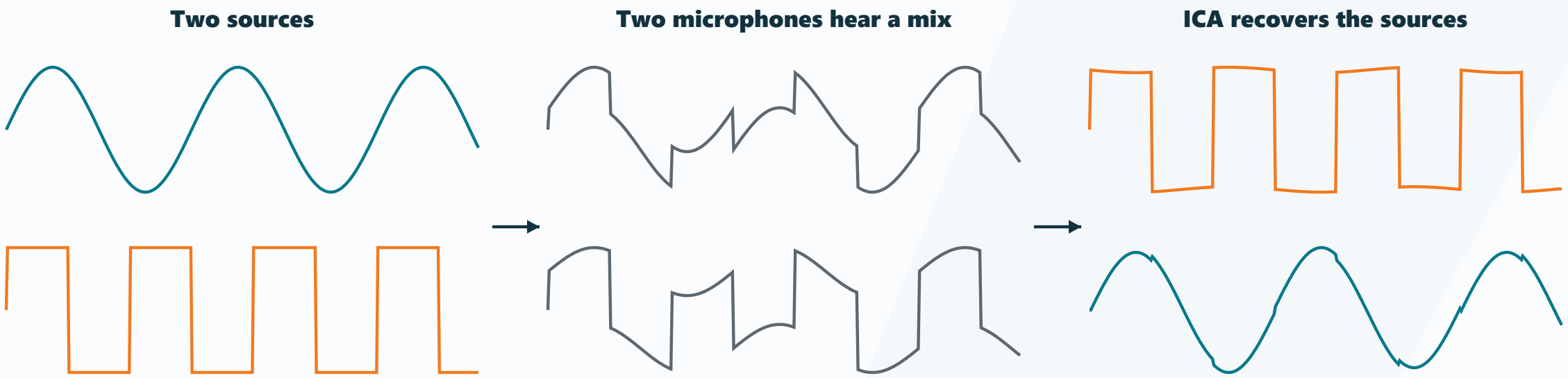
every entry drawn from a normal distribution

Sparse random projection



entries are +s, 0 or −s; most are 0

ICA and the Cocktail Party Problem



PCA asks for directions that are uncorrelated. ICA asks for sources that are independent:

knowing one tells you nothing about the other. It cannot recover their order or sign.

Dictionary Learning: Digits From a Few Atoms



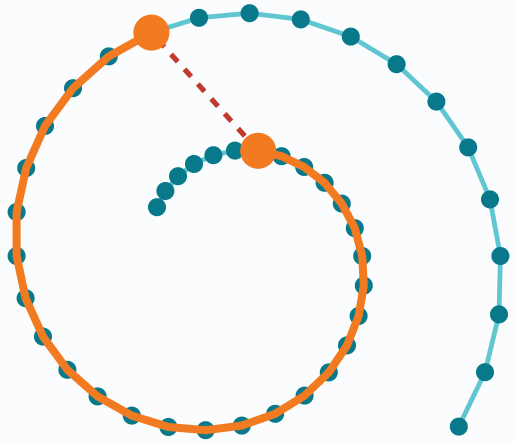
digit | atom 34 | atom 6 | atom 18 | atom 7

The code: 40 numbers, almost all zero



Atoms are learned from the data; each digit uses only a few of them

Isomap and Distance Along the Manifold



Straight line (red): the two points look close

Along the surface (orange): they are far apart

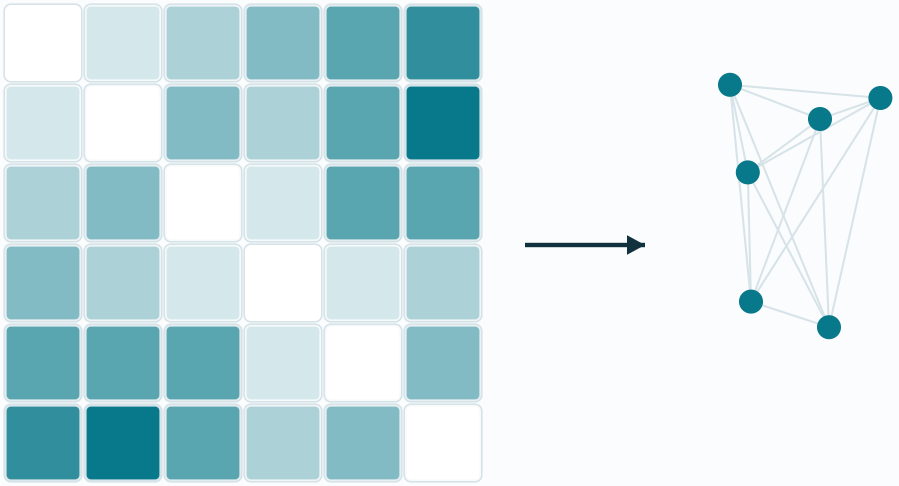
Isomap

1. connect each point to its nearest neighbours
2. distance = shortest path through that graph
3. lay the points out in 2D to keep those paths

```
n_neighbors sets how far "local" reaches
```

MDS and Locally Linear Embedding

MDS: keep every pairwise distance

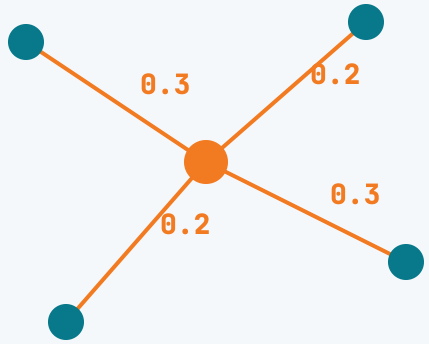


distance matrix, n × n

2D layout, same distances

Cost grows with n^2, so run it on a sample

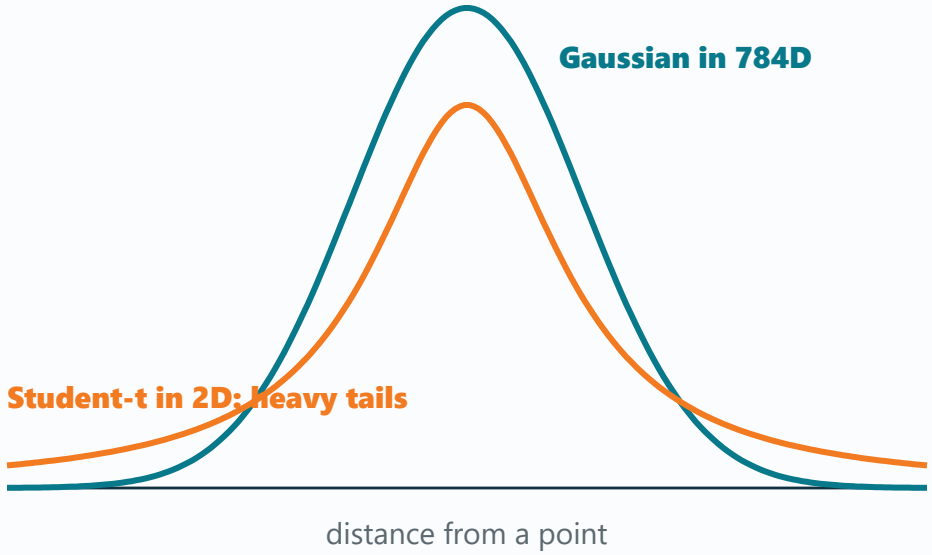
LLE: keep each point's local recipe



each point = weighted mix of its neighbours;
find 2D positions that keep the same weights

Fast, but can fold or squash the layout

How t-SNE Builds a Map



1. In 784D, turn distances into neighbour probabilities with a Gaussian

2. In 2D, use a heavy-tailed Student-t

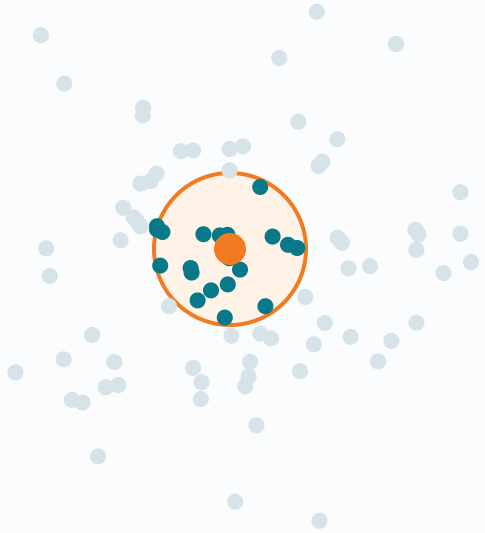
3. Move the 2D points until the two sets of probabilities match

Heavy tails let clusters spread apart in 2D.

Read t-SNE plots with care:
cluster sizes and the gaps between clusters mean little; only who sits next to whom does

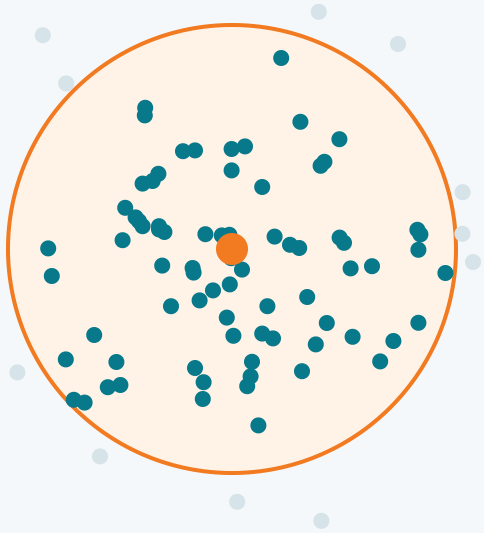
What Perplexity Changes

low perplexity



only the closest few count

high perplexity

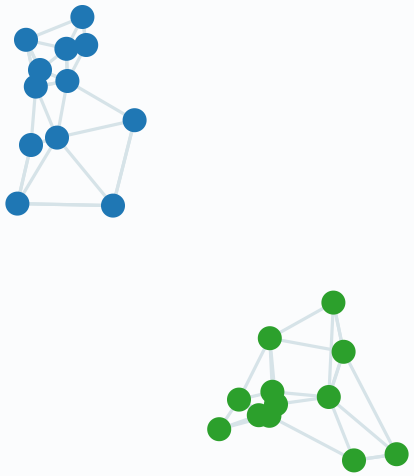


a wide circle of neighbours

Perplexity is roughly the number of neighbours each point pays attention to

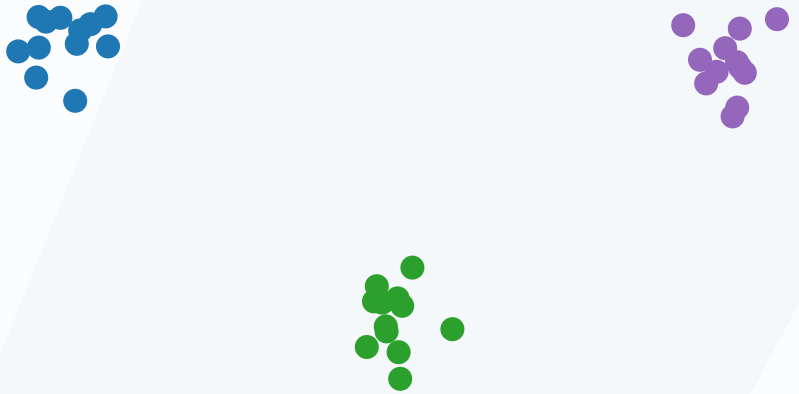
How UMAP Builds a Map

1. a graph of each point's nearest neighbours



edges weighted by how close they are

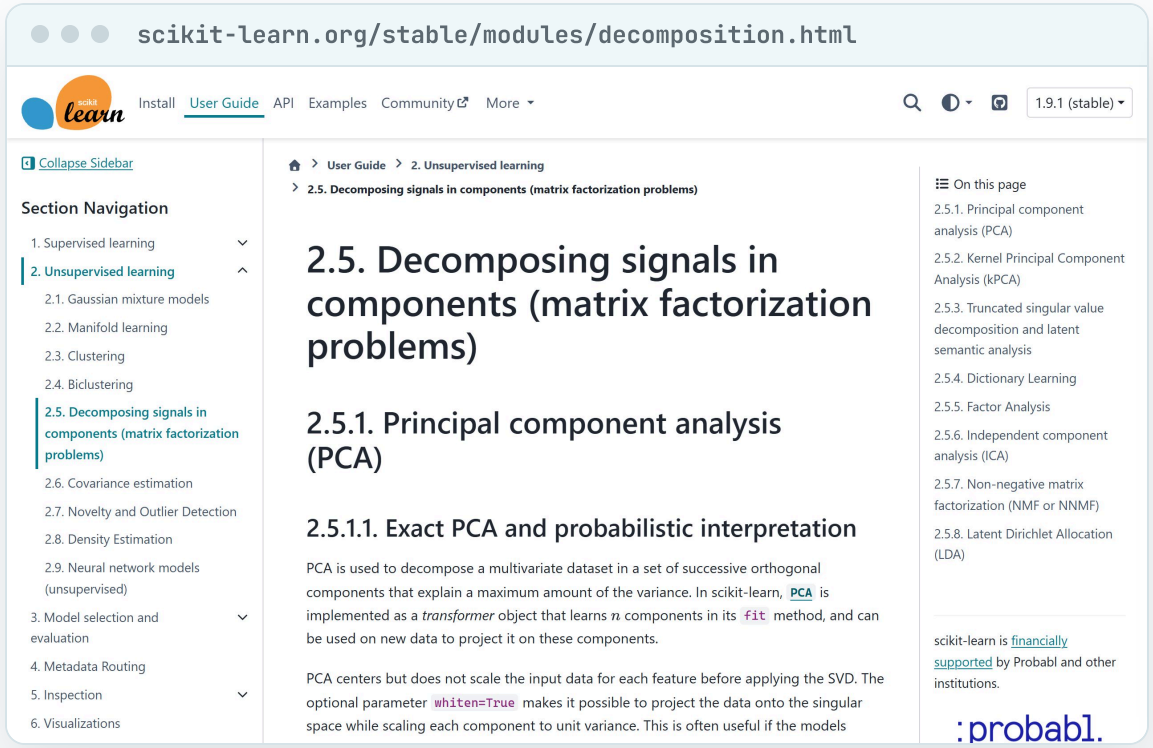
2. a 2D layout that keeps those edges



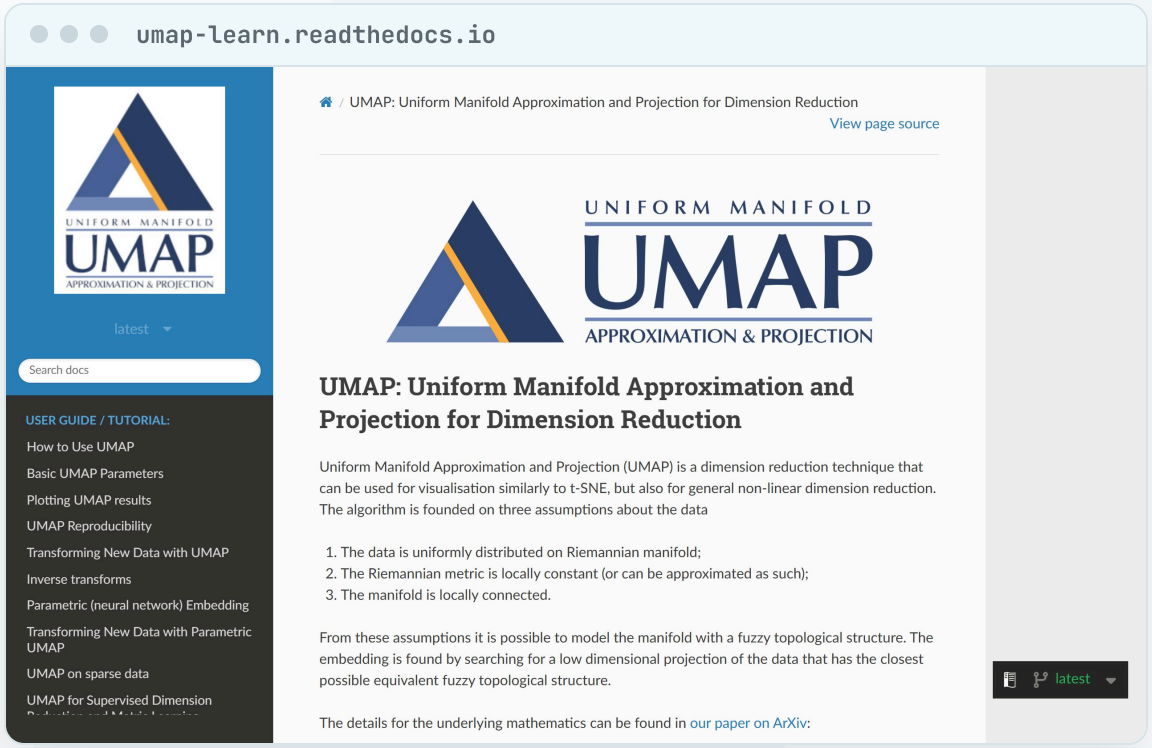
neighbours pull together, others push apart

Faster than t-SNE, keeps more of the big picture, and `transform` places new points on the same map.

scikit-learn and UMAP Docs



scikit-learn: PCA, ICA, dictionary learning and more



umap-learn: the UMAP library and its guides

Each class in this section has a user guide page with the maths behind it.

Choosing a Dimensionality Reduction Method

METHOD	KIND	NEW POINTS	SCALES TO	USE IT FOR
PCA	linear	transform	millions	first step almost always; compression, denoising
Incremental PCA	linear	transform	any size	data that does not fit in memory
Sparse PCA, ICA, dictionary	linear	transform	thousands to fit	readable parts, separated signals
Truncated SVD	linear	transform	millions, sparse	text and ratings matrices
Random projection	linear	transform	millions	fast shrinking before distance-based methods
Kernel PCA, Isomap, MDS, LLE	nonlinear	some	thousands	small data on a curved surface
t-SNE	nonlinear	no	tens of thousands	looking at clusters
UMAP	nonlinear	transform	millions	looking at clusters, features for clustering

SECTION 02 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie